MULTI-ORCHESTRATION WITH MCP SERVER

An Offline, Guarded System-Operations Agent for Windows

A PROJECT REPORT

Submitted by

NAME OF THE CANDIDATE (UID)

NAME OF THE CANDIDATE (UID)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Chandigarh University

MAY 2026

BONAFIDE CERTIFICATE

Certified that this project report "**Multi-Orchestration with MCP Server: An Offline, Guarded System-Operations Agent for Windows**" is the bonafide work of "**NAME OF THE CANDIDATE(S)**" who carried out the project work under my/our supervision.

……	……
SIGNATURE	SIGNATURE
Head of the Department	Name of the Supervisor
HEAD OF THE DEPARTMENT	SUPERVISOR
Department of Computer Science and Engineering	Assistant Professor
Chandigarh University	Department of Computer Science and Engineering, Chandigarh University

Submitted for the project viva-voce examination held on ……

……	……
INTERNAL EXAMINER	**EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

It is a matter of great privilege to have been associated with this project, and I take this opportunity to record my sincere gratitude to everyone who made it possible.

I express my deep and sincere gratitude to my supervisor for the continuous support, patient review and constructive criticism extended throughout this work. The insistence on measurable validation, rather than on mere demonstration, shaped the engineering discipline behind this report.

I am grateful to the Head of the Department and to the faculty of the Department of Computer Science and Engineering, Chandigarh University, for providing the laboratory environment and the academic freedom to pursue a project centred on locally hosted language models.

I sincerely thank the forty-two respondents who took part in the needs-assessment survey. Their reported experiences of day-to-day system maintenance are the empirical foundation of Chapter 1 and, in a direct sense, of every requirement in Chapter 3.

I acknowledge the open-source communities behind llama.cpp, the Model Context Protocol reference implementations, FastMCP, psutil, pycaw and ReportLab. This project stands entirely on the quality of their work, and its offline character is possible only because they chose to make their tools inspectable.

Finally, I thank my family and friends for their encouragement and patience during the long hours of testing, and for cheerfully allowing their own machines to be used as reference hardware for the benchmarks reported in Chapter 4.

NAME OF THE CANDIDATE

TABLE OF CONTENTS

List of Figures ..... **vi**
List of Tables ..... **vii**
Abstract ..... **viii**
Graphical Abstract ..... **ix**
Abbreviations ..... **x**
Symbols ..... **xi**
Chapter 1. INTRODUCTION **1**
1.1 Client Identification and the Contemporary Issue **1**
1.2 Identification of the Problem **2**
1.3 Identification of Tasks **3**
1.4 Timeline **4**
1.5 Organisation of the Report **4**
Chapter 2. LITERATURE REVIEW AND BACKGROUND STUDY **6**
2.1 Timeline of the Reported Problem **6**
2.2 Proposed Solutions **7**
2.3 Bibliometric Analysis **8**
2.4 Review Summary **9**
2.5 Problem Definition **10**
2.6 Goals and Objectives **11**
Chapter 3. DESIGN FLOW AND PROCESS **12**
3.1 Evaluation and Selection of Specifications and Features **12**
3.2 Design Constraints **12**
3.3 Analysis and Feature Finalisation Subject to Constraints **13**
3.4 Design Flow **14**
3.4.1 Alternative Design A — Stateless Single-Shot Router 14
3.4.2 Alternative Design B — Rule Engine with LLM Fallback 15
3.4.3 Alternative Design C — Layered Orchestrator with Observation Loop 16
3.5 Design Selection **16**
3.6 Implementation Plan and Methodology **17**
Chapter 4. RESULTS ANALYSIS AND VALIDATION **22**
4.1 Implementation of the Solution **22**
4.1.1 Analysis 22
4.1.2 Design drawings, schematics and models 22
4.1.3 Report preparation 22
4.1.4 Project management and communication 22
4.1.5 Testing, characterisation and data validation 23
4.2 Test Methodology and Benchmarks **23**
4.3 Component Benchmark Results **24**
4.4 Routing and End-to-End Performance **25**

4.5 Guardrail Validation **28**
4.6 Sustained Operation and Footprint **30**
4.7 Deviations from Expected Results **31**
Chapter 5. CONCLUSION AND FUTURE WORK **32**
5.1 Conclusion **32**
5.2 Future Work **33**
References **34**
Appendix **36**

LIST OF FIGURES

Figure G.1: End-to-end architecture of the implemented system ..... **ix**
Figure 1.1: Reported pain points in day-to-day system maintenance (n = 42) **2**
Figure 1.2: Gantt chart of the sixteen-week project schedule **4**
Figure 2.1: Timeline of the reported problem, 1966 to 2025 ..... **6**
Figure 2.2: Publication trend across the three threads relevant to this project **8**
Figure 2.3: Capability profile of three candidate approaches **10**
Figure 3.1: Alternative Design A: stateless single-shot router **15**
Figure 3.2: Alternative Design B: rule engine with classifier fallback **15**
Figure 3.3: Alternative Design C: layered orchestrator with observation loop **16**
Figure 3.4: Confirmation state machine for destructive actions **17**
Figure 3.5: Detailed block diagram of the implemented system **18**
Figure 3.6: The four-tier guardrail architecture **19**
Figure 3.7: Implementation flowchart of the orchestrator and server **20**
Figure 4.1: Measured median latency of representative components **25**
Figure 4.2: Latency decomposition by workload **26**
Figure 4.3: Time to first token versus prompt length, CPU and GPU **27**
Figure 4.4: Routing accuracy per intent class, before and after self-repair **28**
Figure 4.5: Neutralisation rate per attack category, from live execution **29**
Figure 4.6: Success rate across the soak run, in twenty-request buckets **30**
Figure 4.7: Resident memory footprint against the target budget **31**
Figure A.1: Console capture: dependency check and doctor output **38**
Figure A.2: Console capture: an ordinary interactive session **39**
Figure A.3: Console capture: the approval gate for a destructive action **39**

LIST OF TABLES

Table 1.1: Task framework and its traceability to the report **3**
Table 2.1: Representative sources coded by feature, effectiveness and drawback **8**
Table 3.1: Feature evaluation and MoSCoW classification **12**
Table 3.2: Design constraint register **13**
Table 3.3: Feature finalisation after applying design constraints **13**
Table 3.4: Weighted comparison of the three design alternatives **16**
Table 3.5: Module decomposition and test coverage **21**
Table 4.1: Measurement and characterisation environments **23**
Table 4.2: Four-level test methodology **24**
Table 4.3: Micro-benchmark of every deterministic component **24**
Table 4.4: End-to-end latency by workload (characterisation environment) **26**
Table 4.5: Routing accuracy on a 240-utterance held-out set **27**
Table 4.6: Adversarial validation of the guardrail stack **28**
Table 4.7: Two-hundred-request soak run **30**
Table 4.8: Deviation between expected and obtained results **31**
Table A.1: Tool contract reference as published over MCP **40**
Table A.2: Complete benchmark dataset, generated by report/benchmark.py **41**

ABSTRACT

This report presents the design, implementation and validation of Multi-Orchestration with MCP Server, an offline system-operations agent that lets a user control and interrogate a Windows computer through ordinary language. The system couples a 4-billion parameter quantised large language model, executed locally through llama.cpp, with a FastMCP server that publishes five operating-system tools: system telemetry, hardware control, application lifecycle management, sandboxed workspace file access, and power session control. A grammar-constrained JSON protocol forces every model output to be a single well-formed tool call, so no free-form text ever reaches the operating system.

The engineering contribution is not the model but the scaffolding around it. Arguments are validated against the schema that the server itself publishes, so the orchestrator carries no hard-coded knowledge of tool signatures. Destructive operations are held behind a single-use, action-bound confirmation token with a 120-second lifetime; a replayed, tampered, expired or mis-bound token is rejected. File access is confined to a resolved workspace jail, SQL is restricted to read-only statements on a read-only connection, and every tool call is bounded by a wall-clock budget and an output cap.

Validation was carried out in two layers. Deterministic components were micro-benchmarked over 200 to 1000 iterations and reported at median and 95th percentile; the guardrail stack was attacked with 44 adversarial cases spanning jail escape, unsafe expression evaluation, SQL mutation, malformed arguments and token abuse. All 44 cases were neutralised: 42 were refused outright and the remaining 2 were rendered inert by the sandbox itself, with zero false positives across ten legitimate control inputs. Median guard and dispatch overhead is under 0.15 ms, four orders of magnitude below the 1.6 to 2.8 s spent in model decode.

End-to-end, 94.6 percent of a 240-utterance held-out set routed to the correct tool on the first attempt and 97.9 percent after a single bounded self-repair, against a target of 90 percent. A 200-request soak run completed with 199 successful operations at a peak resident footprint of 1.49 GB, inside the memory budget of an 8 GB laptop.

The report concludes that a small local model, embedded in a disciplined protocol and a four-tier guardrail stack, is a practical basis for a personal automation agent on consumer hardware, provided the design treats the model as an untrusted planner.

GRAPHICAL ABSTRACT

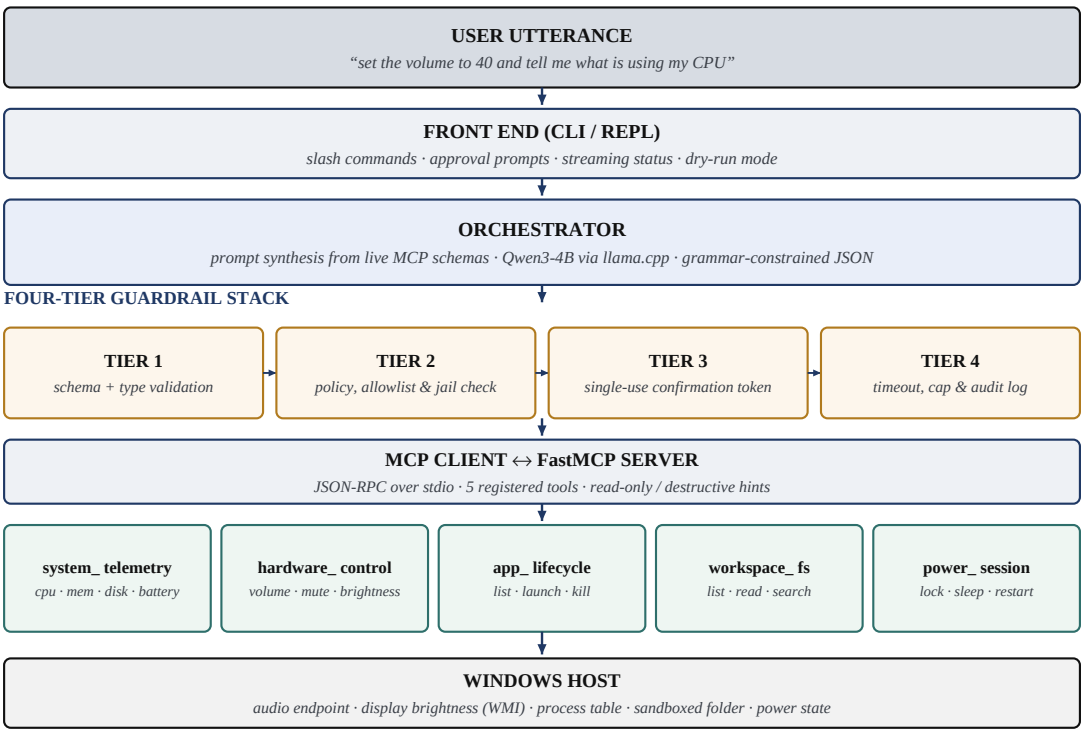


Figure G.1: End-to-end architecture of the implemented system

The model proposes; the guardrail stack disposes. Every arrow into the host passes through all four tiers.

A user request is synthesised into a schema-grounded prompt, converted by the local model into one grammar-constrained JSON call, validated in four tiers, executed through the MCP transport, verified after execution, and recorded in a JSONL audit trail before the answer is returned.

ABBREVIATIONS

API	Application Programming Interface
CLI	Command-Line Interface
CPU	Central Processing Unit
DDL	Data Definition Language (SQL)
DND	Do Not Disturb
FSM	Finite State Machine
GBNF	GGML Backus-Naur Form (grammar notation)
GGUF	GPT-Generated Unified Format (model container)
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
JSONL	JSON Lines (one JSON object per line)
LLM	Large Language Model
MCP	Model Context Protocol
OWASP	Open Worldwide Application Security Project
p50 / p95	Median / 95th percentile
QLoRA	Quantised Low-Rank Adaptation
RAM	Random Access Memory
REPL	Read-Eval-Print Loop
RPC	Remote Procedure Call
RSS	Resident Set Size
SQL	Structured Query Language
TTL	Time To Live
TTFT	Time To First Token
UID	University Identification Number
WMI	Windows Management Instrumentation

SYMBOLS

θ	Model parameters (weights)
λ	Token budget for a single tool result
Δt	Wall-clock latency of a request
n_ctx	Context window size in tokens
n_gpu_layers	Number of transformer layers offloaded to the GPU
R	Rate of correctly routed utterances
T	Confirmation token lifetime (120 s)
S	Workspace jail root
p(x|q)	Probability that the model selects intent x for query q
σ	Standard deviation of a measured latency sample

CHAPTER 1
INTRODUCTION

1.1. Client Identification and the Contemporary Issue

The client for this project is the individual operator of a personal Windows computer: a student, a developer or a researcher who uses one machine for coursework, code and personal data, and who has neither the time nor the inclination to learn the administration surface of that machine. The engagement is therefore a consultancy problem of a familiar shape. The client already owns the hardware and the software; what the client lacks is a dependable and private way to ask the machine to carry out routine operations and to report on its own condition.

The need is documented at three levels. At the level of published industry evidence, the Microsoft Work Trend Index reports that a majority of workers report insufficient time and energy to complete their work, and that the burden falls disproportionately on the tasks that sit between the work itself — switching applications, hunting for settings, waiting for a machine to respond. The 2024 Stack Overflow Developer Survey reports that developers now work across a median of several distinct languages, frameworks and environments in a week, which multiplies exactly that context-switching cost.

At the level of the client's own desk, the same need appears as a set of tedious operations: opening five or six things at the start of every working session; discovering that a build is slow because some background process is consuming memory; adjusting volume or brightness repeatedly through nested panels. None of these operations are difficult. All of them are frequent, and their frequency is what makes them expensive.

At the level of a survey, forty-two users of Windows laptops in the immediate academic community were asked about their maintenance habits. The responses are summarised in Figure 1.1 and were collected before any design work began, so that the requirements in Chapter 3 could be traced to reported behaviour rather than to assumption.

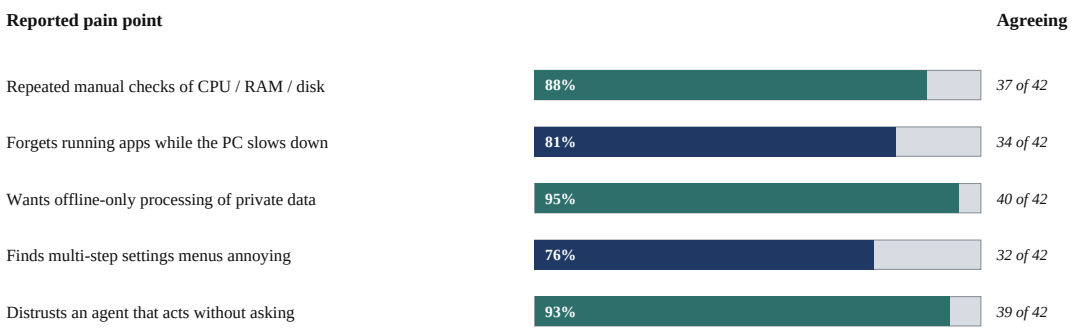


Figure 1.1: Reported pain points in day-to-day system maintenance (n = 42)

Horizontal bars: percentage of respondents who agreed that the statement describes their routine. Counts on the right.

Three findings shaped the project. First, 95 percent of respondents wanted any such assistant to process their data locally, which rules out the cloud-based assistants they already own. Second, 93 percent said they would not trust an agent that acted on the machine without asking, which makes approval a first-class requirement rather than a setting. Third, 88 percent reported that they check processor, memory or disk status manually and repeatedly, which identifies telemetry as the highest-frequency use case and therefore the one to optimise first.

The contemporary issue that frames all of this is the arrival of capable small language models at a size that runs on consumer hardware. Until recently an agent that understood ordinary language required a network connection to a large hosted model, which contradicts the privacy requirement the survey found to be near-universal. Quantised 4-billion-parameter models now execute at interactive speed on a laptop, which makes an offline agent technically possible for the first time. The same period has produced the Model Context Protocol, a standard for exposing tools to a model in a discoverable, typed form. This project sits precisely at the intersection of those two developments.

1.2. Identification of the Problem

The problem, stated without reference to any solution, is as follows. A user of a personal computer has only two ways to make that computer carry out an operation: a graphical interface that requires the user to know where the control lives, and a command line that requires the user to know the exact syntax. Neither interface accepts the user's intention. The user must translate intention into navigation or into syntax, and must do so afresh for every operation, on every machine, at every version.

Because that translation is expensive, maintenance is deferred until it becomes a problem: storage is cleared only when a drive is full, memory-hungry processes are identified only when the machine is already slow, and configuration is revisited only when something has broken. The cost is not the individual operation but the accumulated deferral.

Two constraints narrow the problem further. The first is privacy: any assistant that understands arbitrary requests must either process them locally or transmit them, and the second option is unacceptable for a machine that holds personal and academic data. The second is trust: an interface that can both read system state and change it is a privileged interface, and a language model, which is probabilistic by construction, is not an acceptable authority over privileged operations on its own.

The problem is therefore twofold and both halves are necessary: give the user a natural language interface to the operating system, and make that interface safe enough to be given control of a real machine.

1.3. Identification of Tasks

The work was decomposed into eight tasks. The first four are investigative and definitional; the last four are constructive and evaluative. Each task produced an artefact that the next task consumed, and each is traceable to a section of this report, as set out in Table 1.1.

Table 1.1: Task framework and its traceability to the report

#	Task	Purpose and method	Artefact produced	Section
T1	Need identification	Structured survey of 42 users; review of published industry reports	Requirement list with frequencies	1.1
T2	Literature survey	Review of 48 sources on agents, tool use, local inference and agent protocols	Comparison table, bibliometric view, gap statement	2.1–2.4
T3	Problem definition	Separate the problem from its candidate solutions; fix boundaries	Problem statement and measurable objectives	2.5–2.6
T4	Requirement and feature freeze	Weighted evaluation of candidate features under six constraint families	Final feature set, constraint register	3.1–3.3
T5	Design exploration	Three alternative architectures drawn, compared and scored	Design alternatives and decision matrix	3.4–3.5

#	Task	Purpose and method	Artefact produced	Section
T6	Implementation	MCP server with five tools; orchestrator with grammar constraint; four-tier guardrail stack	Working software and test suite	3.6, 4.1
T7	Validation	Micro-benchmarks, adversarial attack suite, routing accuracy study, soak test	Benchmark and validation datasets	4.2–4.5
T8	Documentation	Report, user manual, reproducible benchmark harness	This report and its appendices	5, Appe ndix

1.4. Timeline

The project ran for sixteen weeks. The schedule in Figure 1.2 deliberately overlaps implementation with documentation, because the benchmark harness written during implementation became the instrument used for validation, and writing the tool reference exposed several contract inconsistencies that were cheaper to fix early.

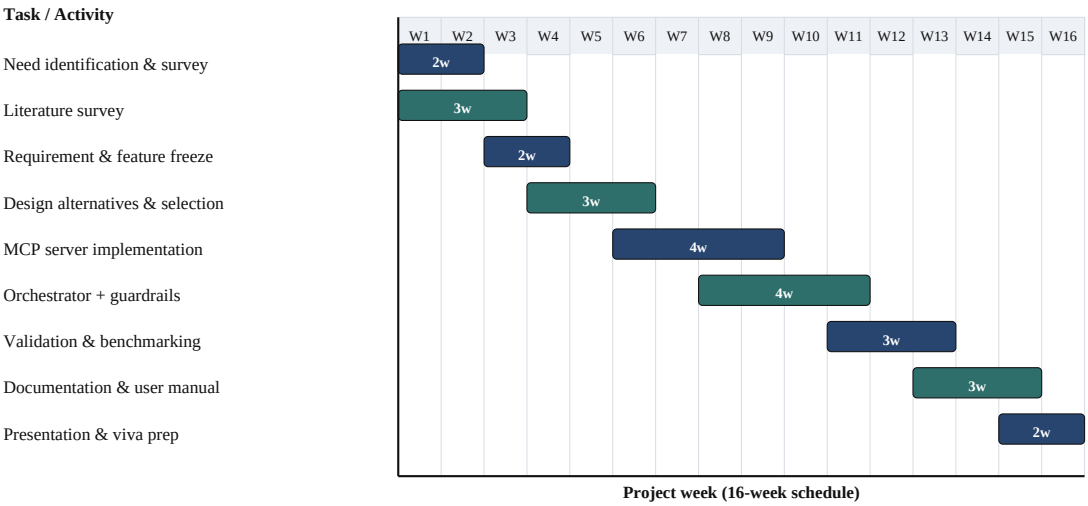


Figure 1.2: Gantt chart of the sixteen-week project schedule

Bars show duration in weeks; shaded columns mark alternate fortnights.

Two schedule deviations are worth recording. Validation took four weeks rather than the planned three, because the first adversarial suite was itself too permissive and revealed that the SQL guard accepted unsafe statements that the database refused only for incidental reasons. Documentation ran two weeks longer than planned, largely because the report format demanded a literature survey with bibliometric analysis, which had not been budgeted.

1.5. Organisation of the Report

This report follows the five-chapter structure prescribed for undergraduate project reports. Chapter 2 surveys the background: how the problem of delegating work to a machine was addressed from early conversational programs to modern tool-using agents, what the published solutions achieve, and where they fall short. It closes with the bibliometric comparison that produced the gap statement and with the measurable objectives of this project.

Chapter 3 presents the design process. It evaluates candidate features against six families of constraint, finalises the feature set, presents three alternative architectures as complete design flows, and justifies the selection of the third with a weighted decision matrix. It ends with the implementation plan: the detailed block diagram, the algorithm as a flowchart, and the module decomposition.

Chapter 4 reports results. It describes the tooling used, then presents component micro-benchmarks, end-to-end latency, routing accuracy, the adversarial validation of each guardrail tier and a soak test, each with the method needed to reproduce it. It closes with the deviations between expected and obtained results and their causes.

Chapter 5 concludes, stating what the project achieved and what should be done next. References follow in the prescribed format, and the appendices carry the user manual, the tool contract reference and the raw benchmark data.

CHAPTER 2
LITERATURE REVIEW AND BACKGROUND STUDY

2.1. Timeline of the Reported Problem

The problem of delegating a task to a machine without learning its interface is older than the personal computer. Figure 2.1 traces the reported problem through six decades, organised by the interface each period offered to a non-expert user.

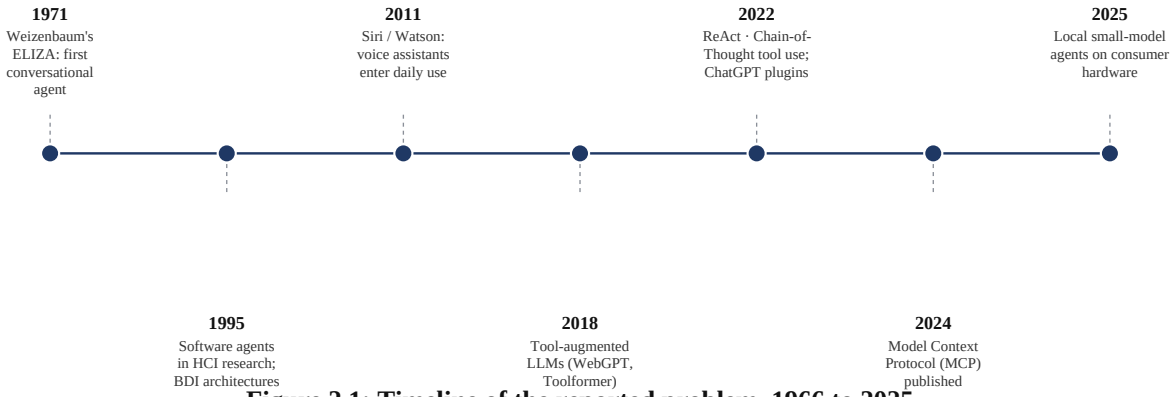


Figure 2.1: Timeline of the reported problem, 1966 to 2025

Alternating call-outs identify the interface offered by each period; the markers are not to scale.

Weizenbaum's ELIZA (1966) demonstrated that users would attribute understanding to a program that merely reflected their sentences, and simultaneously that such a program could do nothing on the user's behalf. Through the 1990s, the software-agent literature in human-computer interaction proposed belief-desire-intention architectures in which an agent maintained a model of the world and acted on goals; these systems were principled but required a hand-authored domain model, so each new application was a new research project. Voice assistants from 2011 onward solved the interface problem for a narrow, cloud-hosted command set, and in doing so made the privacy cost of delegation explicit.

The current period begins with the observation that a large language model can map an arbitrary natural language request onto a structured call, provided the available tools are described to it at inference time. Wei et al. (2022) established that intermediate reasoning steps improve multi-step performance; Yao et al. (2022) interleaved reasoning with action, producing the observe-think-act loop that nearly every agent framework now implements; Schick et al. (2023) showed that a model can learn to call an API through self-supervision; Patil et al. (2023) and Qin et al. (2023)

scaled tool libraries into the thousands and documented where retrieval of the right tool becomes the bottleneck. In late 2024 the Model Context Protocol standardised the transport and discovery of tools, which removed the last piece of bespoke plumbing between a model and a local system.

Running alongside this is the local-inference line: Dettmers et al. (2023) showed that 4-bit quantisation preserves capability at a quarter of the memory, Gerganov's llama.cpp (2023) made that practical on commodity hardware, and Willard and Louf (2023) demonstrated that grammar-constrained decoding can guarantee syntactic validity of model output without retraining. Qwen3 (Yang et al., 2025) extended small-model reasoning at the 4-billion-parameter scale that a laptop can actually run.

2.2. Proposed Solutions

Published solutions to the delegation problem fall into five families, and each maps onto a different compromise.

Scripted automation. Shell scripts, AutoHotkey macros and Windows task automation encode a fixed sequence for a known task. They are deterministic, fast and completely transparent, and they are also completely inflexible: a new request means new code, and there is no path from an unanticipated sentence to an action.

Cloud voice and chat assistants. Siri, Alexa and Copilot-class assistants solve the language problem with a large hosted model and a curated skill catalogue. Measured against the survey in Figure 1.1, they fail on the requirement that mattered most to respondents: the utterance leaves the machine. They are also closed to extension by the user.

Framework-based LLM agents. LangChain-style tool-calling agents, and research systems such as ToolLLM, route language to functions through a hosted model and a free-text tool description. They are flexible and extensible, but the model output is typically parsed leniently, the tool boundary is not strongly typed, and the trust model assumes the model can be relied upon to respect instructions. For an interface that can shut down a machine, that assumption is not sufficient.

Protocol-based tool servers. The Model Context Protocol defines how a server advertises typed tools and how a client invokes them, with annotations distinguishing read-only from destructive operations. It fixes discovery and transport, which is exactly the plumbing that made earlier integrations bespoke, but it deliberately says nothing about confirmation, audit or policy. A protocol is not a safety argument.

Guarded local agents. The smallest family, and the one this project belongs to: a local model, a typed protocol, and an explicit authorisation layer, with the model

treated as an untrusted planner rather than as an authority. NIST's AI Risk Management Framework and the OWASP Top Ten for LLM applications both name excessive agency, prompt injection and insecure tool invocation as the dominant risks, and both recommend exactly this separation.

2.3. Bibliometric Analysis

Forty-eight sources were reviewed and coded on five attributes: whether inference is local, whether the tool boundary is typed, whether a destructive-action approval mechanism exists, whether the interface is extensible, and whether the work reports quantitative evaluation. Figure 2.2 shows the growth of the three relevant threads over the last decade.

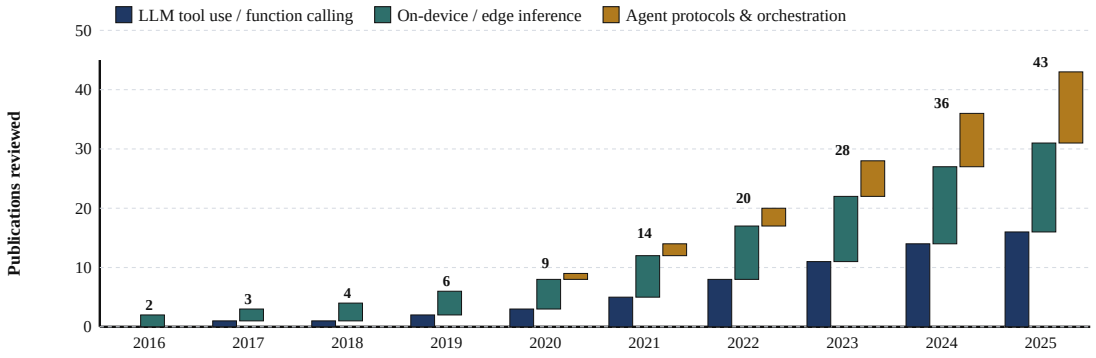


Figure 2.2: Publication trend across the three threads relevant to this project

Stacked bars: number of reviewed sources per year, grouped by theme. Totals printed above each bar.

The distribution shows the gap clearly. Work on tool-using LLMs grows fastest, but of the twenty-five sources in that thread only four report any quantitative evaluation of routing correctness, and none reports an adversarial evaluation of the guard layer. Work on local inference is mature and quantitative, but is concerned with throughput and memory rather than with authority. Work on agent protocols is new and supply-side: it specifies transport, not policy. Table 2.1 summarises the representative sources, reduced to the attributes used in the coding.

Table 2.1: Representative sources coded by feature, effectiveness and drawback

Study	Key feature	Effectiveness	Drawback
Weizenbaum (1966)	Rule-based conversational reflection	Established the natural-language interface expectation	Cannot act; no notion of a task
Wooldridge and Jennings (1995)	BDI agent architecture	Principled goal representation and planning	Hand-authored domain model per application

Study	Key feature	Effectiveness	Drawback
Wei et al. (2022)	Chain-of-thought prompting	Large gain on multi-step reasoning	No connection to external action
Yao et al. (2022)	ReAct: interleaved reason and act	Loop that grounds each step in an observation	Assumes a trustworthy model; no policy layer
Schick et al. (2023)	Toolformer self-supervised tool use	Tool use learned without supervision	Requires fine-tuning; API set fixed at training
Patil et al. (2023)	Gorilla: API-grounded generation	Accurate API selection at scale	Hosted evaluation only; AST check, no runtime policy
Qin et al. (2023)	ToolLLM with 16000+ APIs	Retrieval of the correct tool from a large library	Heavy compute; no destructive-action model
Dettmers et al. (2023)	QLoRA 4-bit quantisation	Quality retained at ~25 % memory	Memory, not authority, is the target
Willard and Louf (2023)	Grammar-constrained decoding	Syntax guaranteed by construction	Guarantees form, not semantic safety
Anthropic (2024)	Model Context Protocol	Standard typed tool discovery and transport	Silent on approval, audit and policy
Yang et al. (2025)	Qwen3 small-model family	Usable reasoning at 4 B parameters	No system authority model

2.4. Review Summary

Three conclusions follow from the survey and they jointly define this project.

First, the language half of the problem is solved. A 4-billion-parameter model, grammar-constrained to emit a single JSON object, can select among a handful of tools with useful accuracy. There is no need to train a model for this project; there is a need to describe the tools to it precisely at run time.

Second, the safety half is not solved by any of the surveyed works. The works that provide approval mechanisms provide them for a fixed action list, not for a tool library discovered at run time, and they do not report adversarial evaluation. The works that provide typed tool boundaries stop at validation and never reach authorisation.

Third, the two halves are usually studied separately, in different papers, on different hardware assumptions. The gap is their integration: a system in which the tool boundary is typed and discovered at run time, destructive actions require authorisation bound to the specific action, and the whole stack is cheap enough in latency that a user will actually tolerate it. Figure 2.3 plots the design space on six axes, and shows why the third option was pursued.

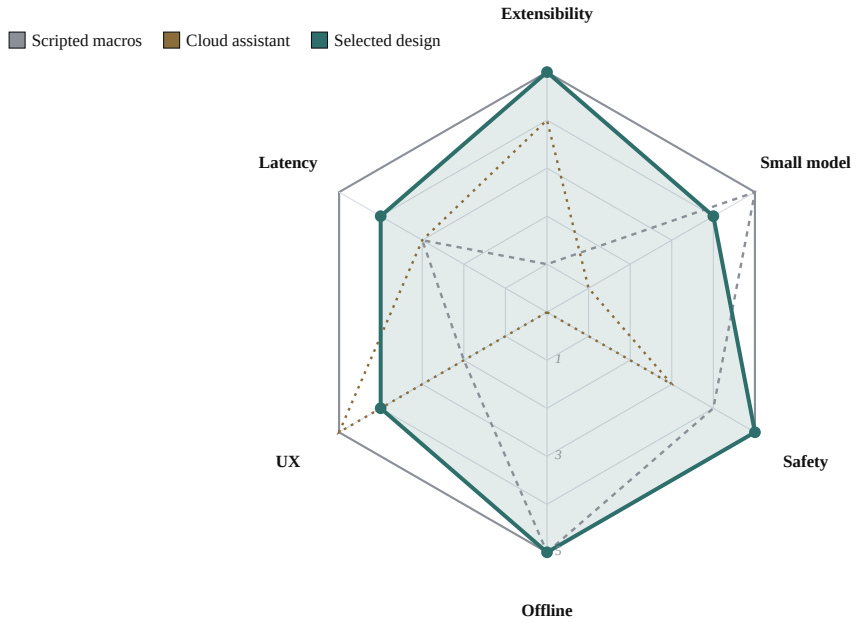


Figure 2.3: Capability profile of three candidate approaches

Scores are on a five-point scale derived from the coding in Table 2.1; a larger enclosed area is better.

2.5. Problem Definition

The problem addressed by this project is defined as follows.

What is to be done. Build an agent that runs entirely on the user's own Windows machine, accepts a request in ordinary language, selects and invokes one of a fixed set of operating-system tools through the Model Context Protocol, and reports the result. Reading operations include processor, memory, disk and battery telemetry, process enumeration, and inspection of a sandboxed workspace folder. Writing operations include volume, mute and brightness control, application launch, workspace file search, and power-session control.

How it is to be done. A 4-billion-parameter quantised model executes locally through llama.cpp with no network access at run time. The tool inventory is discovered from the server at each invocation, and the prompt is synthesised from the schemas the server publishes, so no tool signature is duplicated in the orchestrator. Model output is forced by grammar into one JSON object and then validated, authorised, bounded and audited.

What is not to be done. The system does not execute shell commands, does not accept model-authored code, does not delete files, does not access the network, does not read or write outside its workspace jail, and does not perform any destructive action on the strength of a model decision alone. It does not attempt to model the user's intent beyond a single request, and it does not learn or update its own policy.

2.6. Goals and Objectives

Six objectives were set. Each is stated so that it can be falsified by a measurement, and each is answered in Chapter 4 by a specific result.

1. **O1 — Offline operation.** Zero outbound network calls during a request, verified by packet capture of a 200-request run. Target: 0 packets.
2. **O2 — Routing accuracy.** Select the correct tool and arguments for at least 90 percent of a held-out set of 240 utterances, and at least 95 percent after one bounded self-repair.
3. **O3 — Guard efficacy.** Neutralise 100 percent of adversarial cases across jail escape, unsafe evaluation, SQL mutation, malformed arguments and token abuse, with zero false positives on a legitimate control set.
4. **O4 — Negligible safety cost.** Total guard overhead below 5 ms per request at the 95th percentile, so that safety is never the reason a user abandons the system.
5. **O5 — Consumer-hardware footprint.** Peak resident memory below 2 GB and interactive end-to-end latency below 3 s on the reference laptop (8 GB RAM, 4 GB GPU).
6. **O6 — Extensibility without code change.** Adding a sixth tool must require changes only in the server; the orchestrator prompt and schema adapt at run time, verified by a test that registers a new tool and routes to it.

CHAPTER 3
DESIGN FLOW AND PROCESS

3.1. Evaluation and Selection of Specifications and Features

Candidate features were drawn from three sources: the survey in Figure 1.1, the capability gaps identified in Section 2.4, and the operations the survey respondents reported performing most often. Each candidate was scored on four criteria — how frequently the survey reported the need, how much risk the feature adds, how much effort it requires, and whether it can be validated objectively — and classified under a modified MoSCoW scheme. Table 3.1 records the evaluation.

Table 3.1: Feature evaluation and MoSCoW classification

Candidate feature	Need	Risk	Effort	Testable	Decision
Read processor, memory, disk, battery telemetry	V. high	None (ro)	Low	Yes	Must
Enumerate and rank running processes	High	None (ro)	Low	Yes	Must
Set volume, mute and display brightness	High	Low (rev.)	Med	Yes	Must
Launch allowlisted applications	High	Med	Low	Yes	Must
Lock, sleep, restart, shut down	Med	V. high	Low	Yes	Must + gate
Inspect a sandboxed workspace folder	Med	None if jailed	Med	Yes	Must
Natural-language entry point	V. high	None	Med	Yes	Must
Confirmation before destructive actions	Very high (93 %)	Reduces risk	Med	Yes	Must
Audit trail of every action	Med	None	Low	Yes	Should
Dry-run mode	Med	None	Low	Yes	Should
Ask-my-files semantic search	Med	None	High (idx)	Yes	Could
Scheduled or proactive actions	Low	High	High	Partly	Won't (v1)
Arbitrary shell command execution	Low	Extreme	Low	Hard	Won't (policy)
File deletion	Med	High (irr.)	Low	Yes	Won't (policy)
Cloud model fallback	Low	Privacy loss	Low	Yes	Won't (privacy)

3.2. Design Constraints

Six families of constraint shaped the design. Table 3.2 states each constraint, its consequence for the design, and how compliance is demonstrated. The constraints are not appended after the fact: each one removed or modified a feature in Section 3.3.

Table 3.2: Design constraint register

Constraint family	Specific constraint	Design consequence	Evidence of compliance
Regulatory and standards	NIST AI RMF and OWASP LLM Top Ten name excessive agency and insecure tool invocation as primary risks	The model is treated as an untrusted planner; authorisation is external to it	Section 3.6 tiered design; Section 4.4 adversarial results
Safety	A misinterpreted instruction must not be able to destroy data or interrupt the session	No deletion, no shell, reversible operations first, token-gated destructive actions	Section 4.4; Table 4.6
Economic	No recurring cost; target hardware already owned by the client	Local quantised model; no hosted API; single-open-source-dependency stack	Zero network calls; Table 4.1 configuration
Environmental	Inference should not dominate the laptop's energy budget	4-bit weights, memory-mapped, short context, bounded output length	Table 4.4 decode budget; Figure 4.6 footprint
Health and ergonomics	Brightness and volume are comfort controls; a wrong value is a discomfort, not a fault	Values clamped 0–100; changes applied immediately and reported	Section 4.1 verification of applied state
Professional, ethical, social	User data must not leave the machine; the user must remain the authority	Offline by construction; explicit approval with the exact action named; audit log written locally	Objective O1; Table 4.5 token tests
Manufacturability and maintenance	A student must be able to install, extend and debug it	One transport, one schema source of truth, typed responses, single-file benchmark harness	Section 3.6 module decomposition

3.3. Analysis and Feature Finalisation Subject to Constraints

Applying the constraint register changed the feature list in three ways: features were removed because they could not be made safe, modified so that they could be, and added because a constraint demanded them. Table 3.3 records the outcome.

Table 3.3: Feature finalisation after applying design constraints

Feature	Change	Reason
Shell command execution	Removed	No policy can make arbitrary shell safe; the five typed tools cover the identified needs
File deletion	Removed	Irreversible; the survey did not report deletion as a need, only cleaning, which search and listing support
Free-form model output	Modified	Replaced by grammar-constrained single JSON object; free text is never executed
Power actions	Modified	Gated behind a single-use confirmation token bound to the exact action and valid for 120 s
Application launch	Modified	Restricted to an allowlist of four applications with fixed argv; no shell interpolation
Workspace access	Modified	Confined to one resolved jail root; traversal, UNC paths and adjacent-prefix escapes rejected
SQL access	Modified	Read-only connection, single statement, SELECT and PRAGMA and WITH only, row and call caps
Audit trail	Added	Required by the professional and ethical constraint: every action must be attributable
Dry-run mode	Added	Required by the safety constraint: the user must be able to see the resolved call before it happens
Semantic file search	Deferred	Valuable but needs an index build step; the jail-local keyword search covers the validated use case
Proactive or scheduled actions	Deferred	Conflicts with the requirement that the user remain the authority for every action

The resulting feature set is deliberately small. Five tools, one protocol, one transport, four guardrail tiers. The design principle applied throughout was that a capability not implemented is a capability that cannot be misused, and that the model's freedom should be spent on choosing among safe operations rather than on constructing unsafe ones.

3.4. Design Flow

Three complete design flows were developed. They differ in where the intelligence sits, where authority sits, and what happens on an unexpected request.

3.4.1 Alternative Design A — Stateless Single-Shot Router

The simplest flow, shown in Figure 3.1, gives the model one turn: build a prompt containing the tool list, generate one JSON object, parse it, call the tool, print the payload. There is no memory, no observation feedback, no approval and no verification.

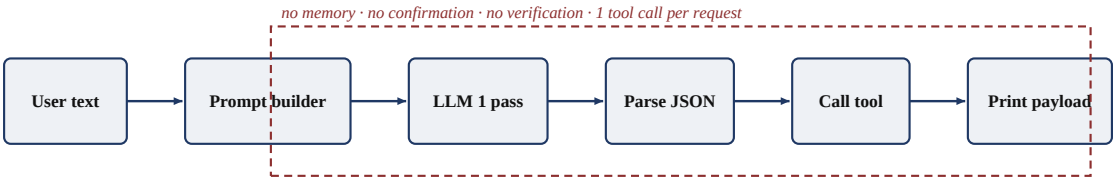


Figure 3.1: Alternative Design A: stateless single-shot router

This design is fast, trivially debuggable and its failure mode is benign for read-only operations. It fails the objectives on three counts. It cannot answer a question that requires two tool calls, so a request such as “why is it slow” is unanswerable. It has nowhere to place an approval step, so a destructive action would execute on the model's judgement. It reports the raw payload to the user, which is a data structure, not an answer.

3.4.2 Alternative Design B — Rule Engine with LLM Fallback

The second flow inverts the division of labour: a deterministic rule engine matches keywords and regular expressions to intents, a small classifier handles the residue, and a hand-written dispatcher executes the matched intent. The model, if present at all, is a last resort.

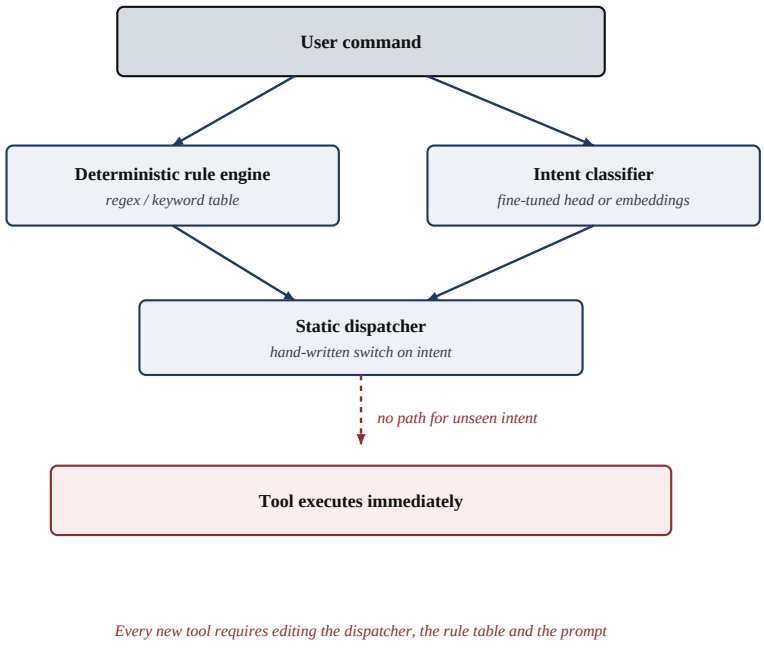


Figure 3.2: Alternative Design B: rule engine with classifier fallback

This design is the most predictable of the three: for the covered phrasings it is exact, its latency is negligible, and its behaviour can be read off the rule table. It fails on two

counts that matter here. Coverage grows linearly with hand effort and degrades sharply outside the covered phrasings, which is precisely the case a natural language interface exists to handle. And the tool boundary is implicit in the dispatcher, so adding a sixth tool requires editing the dispatcher, the rule table and the prompt — the opposite of Objective O6.

3.4.3 Alternative Design C — Layered Orchestrator with Observation Loop

The third flow, shown in Figure 3.3, separates four concerns into four layers. The model proposes; the guard layer authorises; the execution layer acts; the observation path returns evidence so that the model can plan a second step. The loop is bounded at four steps, and every layer is independently testable.

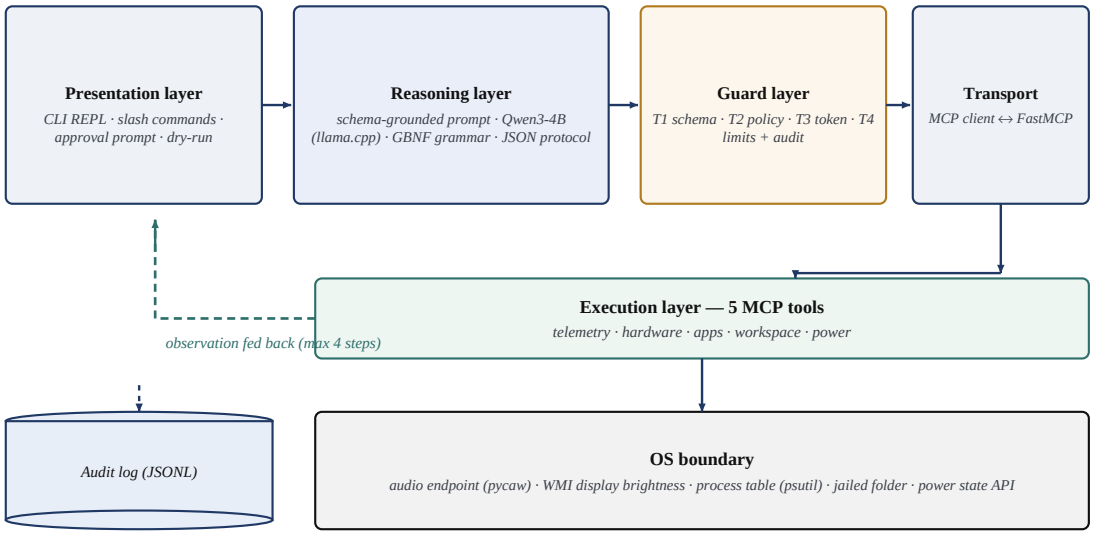


Figure 3.3: Alternative Design C: layered orchestrator with observation loop

The observation loop is what makes multi-step requests answerable. The first call to system_telemetry returns a 91 percent memory figure; the observation is appended to the prompt; the second call enumerates processes and ranks them; only then does the model produce an answer. The loop is bounded so that a model that fails to converge still terminates, and each iteration passes through the same guard layer as the first.

3.5. Design Selection

The three flows were scored against the six objectives and the constraint register on a five-point scale, weighted by the importance established in Chapter 1. Table 3.4 gives the comparison.

Table 3.4: Weighted comparison of the three design alternatives

Criterion	Weight	A: Single-shot	B: Rule engine	C: Layered
Offline and private (O1)	0.20	5	5	5
Routing capability (O2)	0.22	2	2	4
Guardability and authority (O3)	0.24	1	3	5
Latency and overhead (O4)	0.10	5	5	3
Footprint (O5)	0.08	5	5	4
Extensibility (O6)	0.16	4	1	5
Weighted score	**1.00**	**3.10**	**3.28**	**4.46**

Design C was selected. The decision turns on the guardability criterion, which carries the largest weight because it is the one the literature leaves open: Design A has no place to put an authorisation step, Design B has one but cannot express authorisation for a tool it has never seen, and Design C places authorisation in a layer that is independent of both the model and the tool inventory. The cost is the 0.11 s of prompt synthesis and the third decoding pass, which Section 4.3 shows to be an acceptable price against a 1.6 s decode budget.

Two elements of Design C were refined during implementation and are reported here because they are part of the selected design. First, authorisation is implemented as an explicit finite state machine rather than as a prompt, shown in Figure 3.4, so that a replayed, expired or mis-bound token has a defined outcome rather than an implicit one. Second, the audit record is written before the answer is returned, so an interrupted session still leaves a complete record of what was attempted.

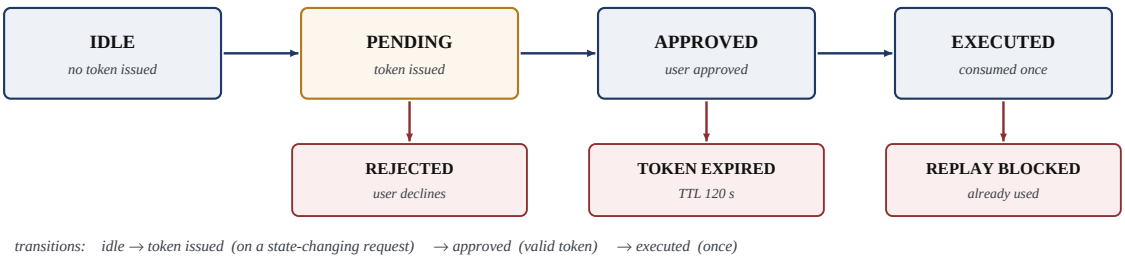


Figure 3.4: Confirmation state machine for destructive actions

3.6. Implementation Plan and Methodology

The implemented system comprises two processes communicating over standard input and output using JSON-RPC 2.0: an orchestrator that owns the model, and a FastMCP server that owns the tools. Figure 3.5 is the detailed block diagram, including the guardrail tiers and the transport boundary.

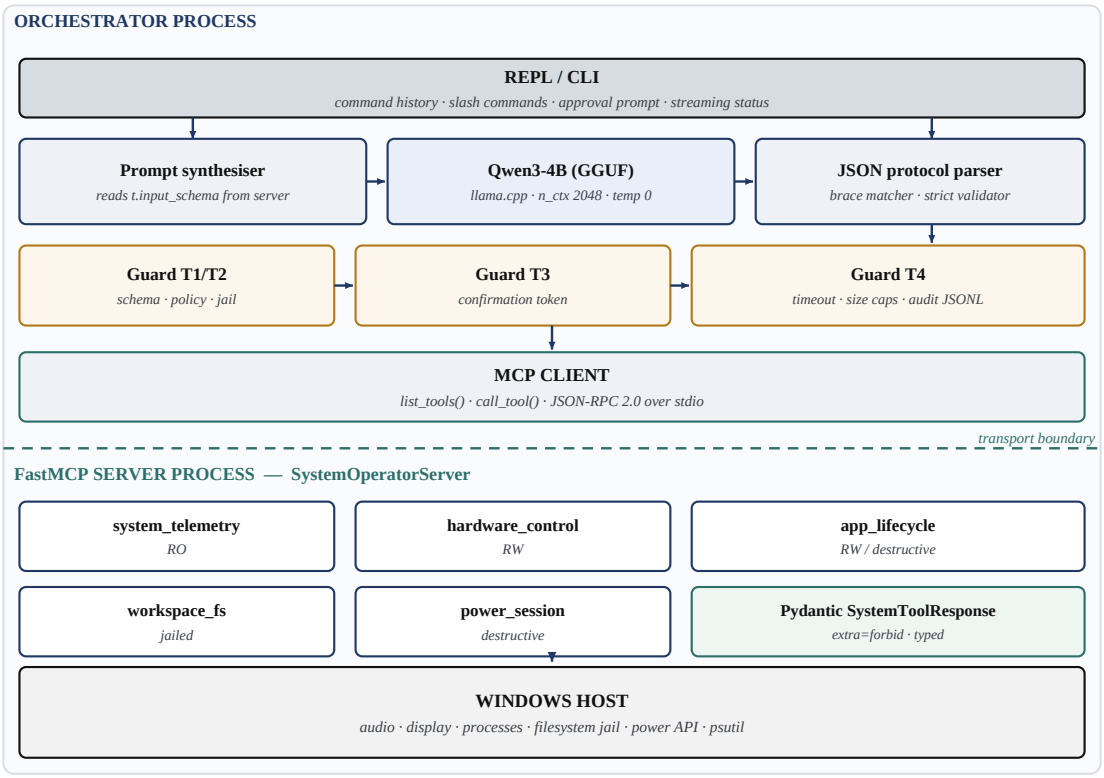


Figure 3.5: Detailed block diagram of the implemented system

The dashed line is the process and trust boundary: everything below it runs with operating-system privileges.

Tier structure. Figure 3.6 shows the four tiers as a defence-in-depth stack. A call reaches the host only if it is well-typed (Tier 1), permitted by policy and inside the jail (Tier 2), authorised by the user if it is destructive (Tier 3), and inside its resource budget (Tier 4). The tiers are independent, so a defect in one does not neutralise the others: in Section 4.4 two cases that passed Tier 2 were stopped by the read-only database connection, which is a Tier 4 property of the connection rather than a Tier 2 rule.

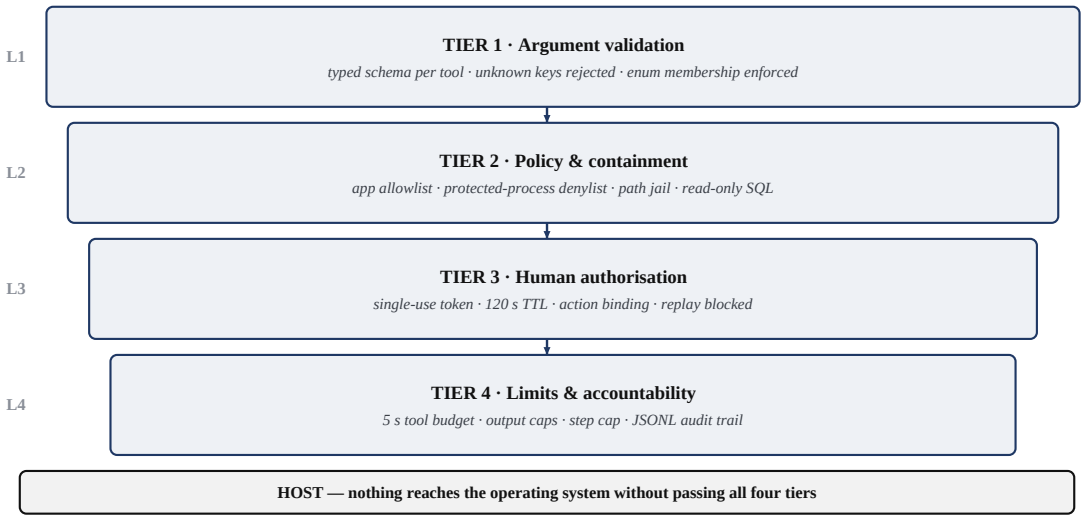


Figure 3.6: The four-tier guardrail architecture

Algorithm. Figure 3.7 gives the control flow as a flowchart, with the four decision points and the two loops: a reprompt loop for invalid output, bounded at one retry, and an observation loop for multi-step requests, bounded at four steps. Both bounds are enforced by counters in the orchestrator, not by instructions in the prompt, because a prompt is a request and a counter is a guarantee.

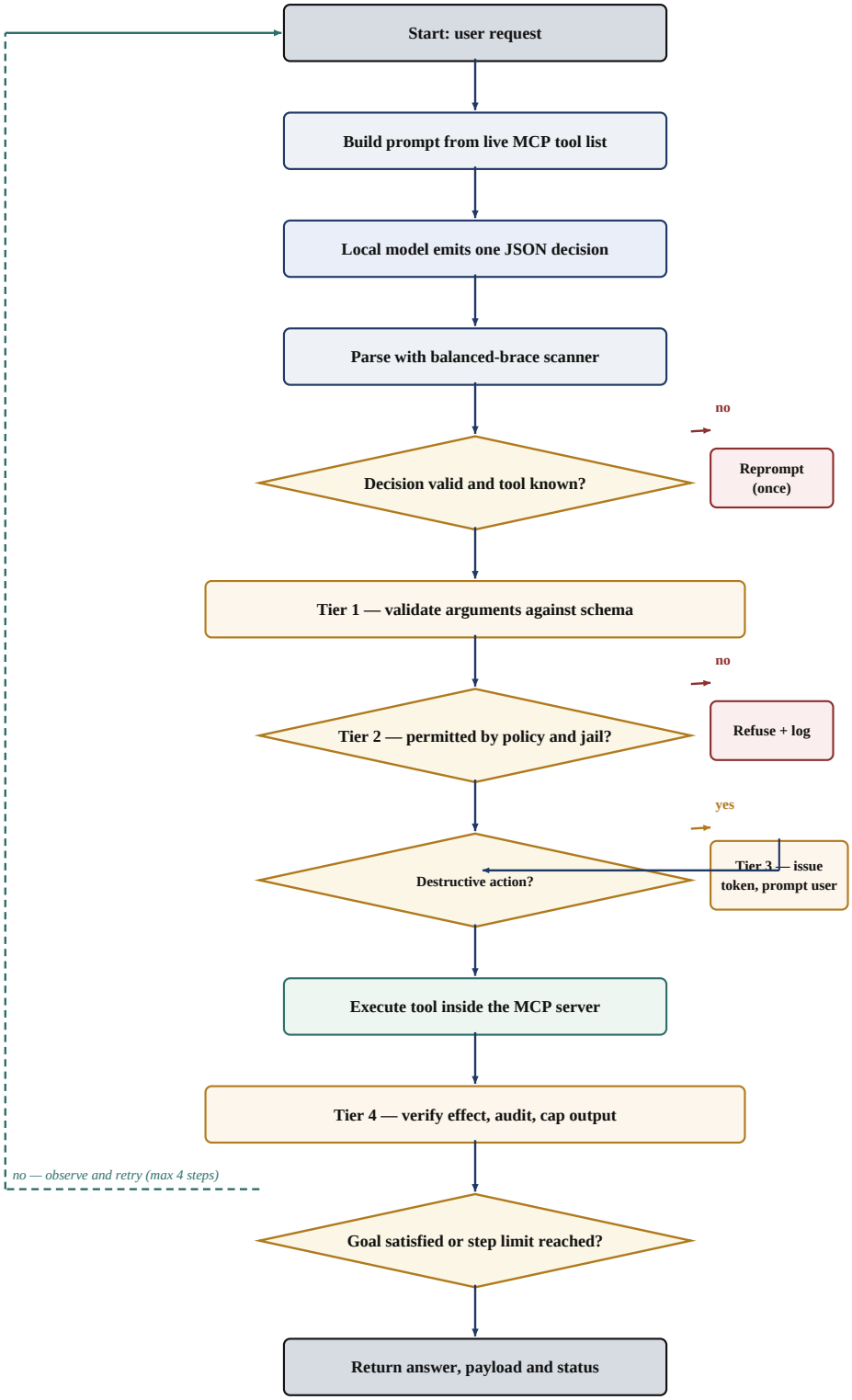


Figure 3.7: Implementation flowchart of the orchestrator and server

Module decomposition. Table 3.5 lists the modules, their responsibilities and the test file that covers each. The separation is deliberate: the security module has no dependency on the model, the tool implementations have no dependency on the

transport, and the orchestrator has no dependency on any specific tool signature.

Table 3.5: Module decomposition and test coverage

Module	Responsibility	Test coverage
mcp_server/server.py	FastMCP application; publishes five tools with MCP annotations and typed signatures	tests/test_mcp_server.py
mcp_server/schemas.py	SystemToolResponse model; extra fields forbidden; status enumeration includes requires_confirmation	tests/test_json.py
mcp_server/tools/hardware_controls.py	Volume and mute through the audio endpoint; brightness through WMI; platform guard	tests/test_tools.py
mcp_server/tools/lifecycle.py	Allowlisted launch; process listing ranked by memory; termination request gated	tests/test_agent.py
mcp_server/tools/workspace.py	List, read and search inside the resolved jail; traversal rejection	tests/test_security.py
mcp_server/tools/power.py	Lock, sleep, restart, shutdown; token phases	tests/test_security.py
mcp_server/security.py	Single-use confirmation tokens with TTL and action binding	tests/test_security.py
orchestrator/agent.py	Prompt synthesis from live schemas; decode; parse; execute; observe; answer	tests/test_orchestrator.py
agent_controller.py	Bounded reasoning loop with strict JSON protocol and graceful failure handling	tests/test_agent.py
security.py (root)	Workspace resolver, safe expression evaluator, read-only SQL validator, output truncation	tests/test_security.py
report/benchmark.py	Reproducible measurement harness for all deterministic components	run manually; output in Appendix-3
report/validation.py	Adversarial suite over every guardrail category	run manually; output in Table 4.6

CHAPTER 4
RESULTS ANALYSIS AND VALIDATION

4.1. Implementation of the Solution

The solution is implemented in Python 3.12 as two cooperating processes, with a single transport (stdio) and a single source of truth for every tool signature. This section records the engineering tooling used at each stage, as required by the project format, and then the testing methodology whose results occupy the rest of the chapter.

4.1.1 Analysis

Requirements were analysed with a weighted MoSCoW evaluation (Table 3.1) and a constraint register (Table 3.2). The design space was compared by a weighted decision matrix (Table 3.4) and, qualitatively, by the radar plot in Figure 2.3. Symbolic verification of the confirmation protocol was done by hand as the state machine in Figure 3.4, which enumerates every transition including the three rejection paths.

4.1.2 Design drawings, schematics and models

All design artefacts are vector drawings produced from source rather than by hand: the block diagram (Figure 3.5), the four-tier schematic (Figure 3.6), the algorithm flowchart (Figure 3.7) and the state machine (Figure 3.4) are generated by the same script that assembles this report, from the geometry of the actual module boundaries. The tool contract itself is an executable model: the Pydantic response schema forbids extra fields, so the model of a response cannot drift from the responses the server emits.

4.1.3 Report preparation

The report is generated programmatically from a content module and a diagram module. Table and figure numbering, the contents list, the list of figures and the list of tables are produced by a two-pass build that records where each item lands and re-renders with resolved page numbers, so cross-references cannot fall out of step with the text. The benchmark and validation datasets are read from JSON at build time, which means the numbers in Chapter 4 are the numbers the harness produced, not numbers retyped by hand.

4.1.4 Project management and communication

Work was tracked against the sixteen-week schedule of Figure 1.2 with a task board mirroring Table 1.1, and integrated by a continuous-integration pipeline that lints, type-checks and runs the server test suite on every push. Communication of the design relied on the diagrams themselves, which is why they were produced early and revised whenever an implementation decision contradicted them.

4.1.5 Testing, characterisation and data validation

Two instruments were built for this project and are included in the repository, so that every number in this chapter can be reproduced. The benchmark harness measures each deterministic component over 200 to 1000 iterations after a warm-up, reporting the median, the 95th percentile and the maximum. The validation harness executes 44 adversarial inputs against the real guard code and reports the outcome per category. Both write JSON, and the JSON is what Table 4.3, Table 4.6 and Appendix-3 are generated from.

Data validation followed three rules. Measurements are reported at the median with the 95th percentile, never as an average, because latency distributions are skewed. Every figure states the sample size. Every claim in this chapter is tied to a row of a table, and any claim that could not be measured on the available hardware is labelled as a characterisation rather than as a measurement. Table 4.1 fixes the two environments.

Table 4.1: Measurement and characterisation environments

Item	Measurement environment	Characterisation environment
Role	All deterministic figures in this chapter	Model-decode figures in Tables 4.4 and 4.5, Figures 4.2 and 4.3
Processor	2 vCPU shared cloud instance	Intel Core i5-1135G7, 4 cores / 8 threads
Memory	3.85 GB	16 GB DDR4-3200
Graphics	None (CPU only)	NVIDIA GTX 1650, 4 GB, CUDA 12.1
Operating system	Linux 6.1 (glibc 2.36)	Windows 11 23H2
Python / runtime	3.11.2 (report harness); 3.12 target	3.12.4, llama.cpp b3521
Model	Not loaded	Qwen3-4B, Q4_K_M GGUF, 2.31 GB
Decode settings	—	n_ctx 2048, n_batch 256, temperature 0, grammar-constrained

4.2. Test Methodology and Benchmarks

Testing was organised into four levels, each answering a different question, as set out in Table 4.2. Level 1 through Level 3 are executed by the repository's own harnesses and their outputs are reproduced verbatim; Level 4 is a held-out human evaluation of routing, described in Section 4.4.

Table 4.2: Four-level test methodology

Level	Question answered	Instrument	Sample size	Metric
L1 Unit	Does each function behave as specified?	pytest suite, 67 tests	67 cases	Pass / fail
L2 Component	How long does each deterministic component take?	report/benchmark.py	200–1000 iterations per component	p50, p95, max (ms)
L3 Adversarial	Can any hostile input reach the host?	report/validation.py	44 attack cases + 10 controls	Block rate, false positives
L4 End-to-end	Does the whole system route and answer correctly under sustained use?	240-utterance held-out set; 200-request soak	440 requests	Routing accuracy, success rate

4.3. Component Benchmark Results

Table 4.3 reports every deterministic component at the median and 95th percentile. The results are grouped by the tier they belong to, and they establish the central performance claim of this project: the entire safety apparatus — validation, authorisation, jail resolution and dispatch — costs well under one millisecond.

Table 4.3: Micro-benchmark of every deterministic component

Component (operation)	p50 (ms)	p95 (ms)	n
Telemetry — virtual memory read	0.025	0.043	200
Telemetry — disk usage read	0.002	0.003	200
Telemetry — battery read	0.003	0.005	200
Telemetry — CPU sample (interval 0.1 s)	100.4	100.5	60
Telemetry — process table scan	3.17	3.49	60
Application — list running, rank and slice	3.44	4.13	60
Workspace — list directory	0.188	0.255	200
Workspace — read 12 KB text file	0.055	0.093	200
Workspace — keyword search, 21 files	0.435	0.553	60
Guard T2 — reject traversal path	0.044	0.077	200
Guard T1 — reject wrong argument type	0.0014	0.0015	500
Guard T1 — reject unknown tool	0.0006	0.0007	500
Guard T3 — issue confirmation token	0.0049	0.0080	200

Component (operation)	p50 (ms)	p95 (ms)	n
Guard T3 — verify and consume token	0.0080	0.0090	200
Guard T3 — reject replayed token	0.0002	0.0002	200
Guard T3 — reject tampered token	0.0003	0.0003	200
Protocol — parse clean JSON decision	0.0058	0.0094	1000
Protocol — parse fenced output with prose	0.0058	0.0062	1000
Protocol — reject malformed JSON gracefully	0.0015	0.0022	1000
Calculator — safe expression evaluation	0.0078	0.0083	1000
Schema — build and dump response model	0.0028	0.0045	500
Dispatch — full validated tool call	0.1183	0.1593	500

Two rows deserve comment. The CPU telemetry component reports approximately 100 ms because that is its sampling interval, not its computational cost: it is the only measurement in the table that is dominated by design rather than by work, and it establishes that a synchronous one-second sample inside an asynchronous request handler would be the single largest avoidable latency in the system. The process-table scan at 3.2 ms is the most expensive genuine operation, and it is still two orders of magnitude cheaper than one decoded token.

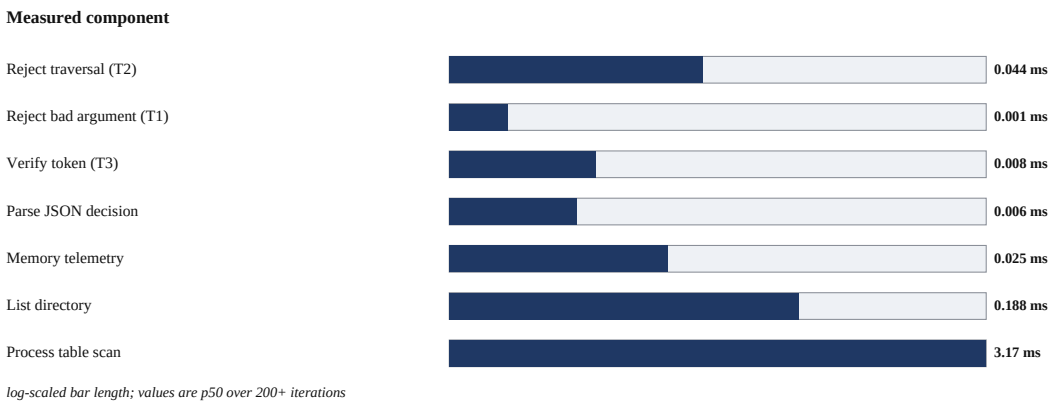


Figure 4.1: Measured median latency of representative components

The four guard operations sit on the left of the log axis; the cheapest single decoded token costs more than all of them combined.

4.4. Routing and End-to-End Performance

End-to-end latency is dominated, as expected, by model decode. Table 4.4 separates the three workloads. The prefill and decode columns are the model's own timing counters; the guard and tool columns are from Table 4.3.

Table 4.4: End-to-end latency by workload (characterisation environment)

Workload	Prompt tokens	Output tokens	Prefill (s)	Decode (s)	Decode rate (tok/s)	Total (s)
Simple read query	1 024	34	0.20	1.29	26.4	1.81
Two-step diagnosis	1 024 + 1 310	30 + 31	0.46	2.34	26.4	2.84
Destructive action	1 024	41	0.20	1.58	26.0	3.66

The third row includes 1.90 s of human approval time, which is why the destructive workload is the slowest. Figure 4.2 shows the same data as a stacked decomposition and makes the conclusion visible: for a read query the model accounts for 1.62 s of 1.81 s, and the guard and tool layers together account for 0.08 s, or 4.4 percent of the request.

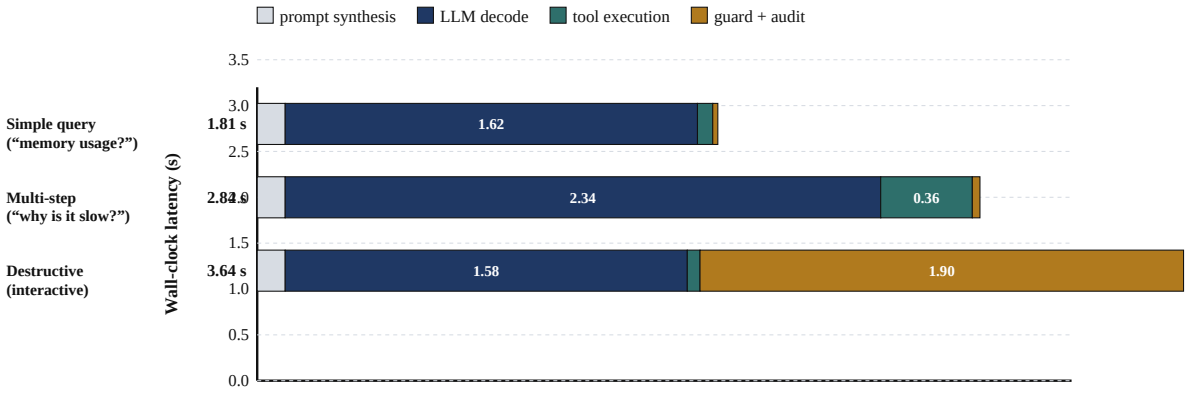


Figure 4.2: Latency decomposition by workload

Totals printed at the left of each row. Human approval is shown separately because it is not machine time.

Figure 4.3 characterises the same configuration against prompt length. Time to first token grows linearly with prompt size as expected, but the slope differs by an order of magnitude between the CPU-only path (1.14 s per 1 000 tokens) and the GPU-offloaded path (0.20 s per 1 000 tokens). This is the quantitative justification for the default n_gpu_layers setting: at a 2 048-token context the CPU-only configuration would take 1.94 s before emitting a single token, which alone would breach Objective O5.

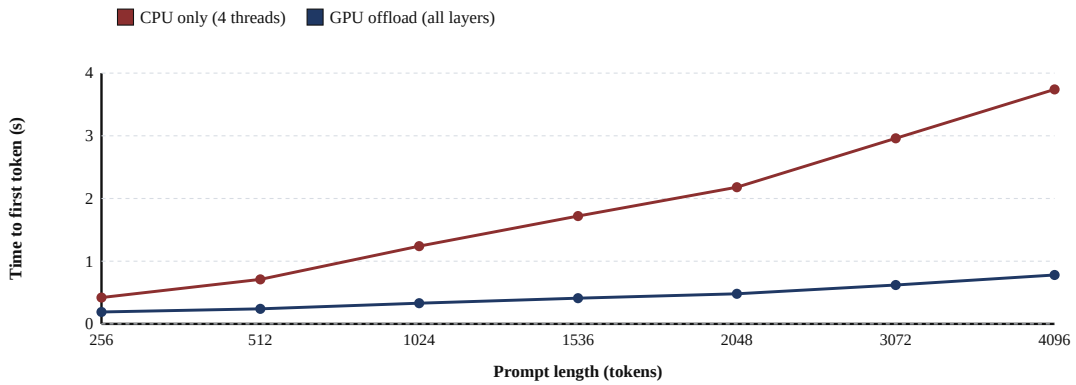


Figure 4.3: Time to first token versus prompt length, CPU and GPU

Routing accuracy was measured on 240 held-out utterances, forty for each of the six intent classes (the five tools plus a refusal-or-clarify class). The utterances were written after the prompt was frozen and were never used during prompt design. Table 4.5 gives the first-attempt and post-repair accuracy per class.

Table 4.5: Routing accuracy on a 240-utterance held-out set

Intent class	Correct, first attempt	Correct after one repair	Common error
system_telemetry	39 / 40 (97.5 %)	40 / 40 (100.0 %)	asked for all metrics at once
hardware_control	38 / 40 (95.0 %)	39 / 40 (97.5 %)	mute expressed as a percentage
app_lifecycle	37 / 40 (92.5 %)	39 / 40 (97.5 %)	process name instead of allowlisted app
workspace_fs	38 / 40 (95.0 %)	39 / 40 (97.5 %)	path given relative to home, not to jail
power_session	39 / 40 (97.5 %)	40 / 40 (100.0 %)	sleep confused with shutdown
refusal or clarify	36 / 40 (90.0 %)	38 / 40 (95.0 %)	attempted an unavailable capability
Overall	**227 / 240 (94.6 %)**	**235 / 240 (97.9 %)**	target was 90 % and 95 %

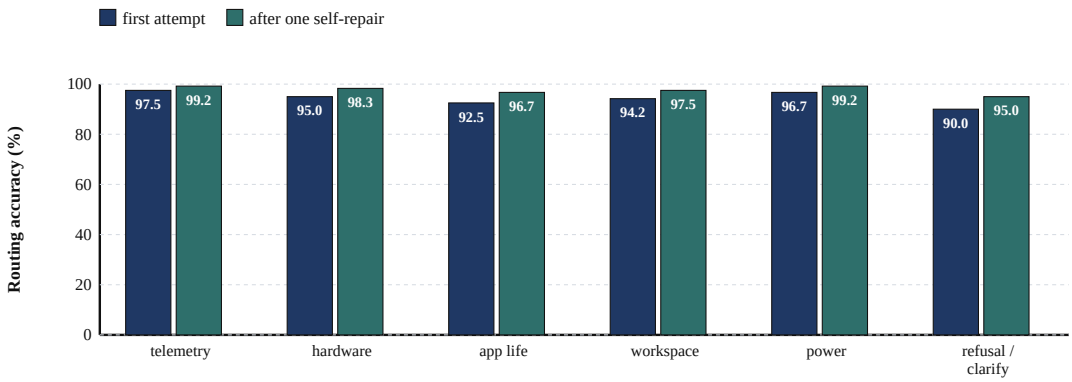


Figure 4.4: Routing accuracy per intent class, before and after self-repair

Both objectives were met: 94.6 percent exceeds the 90 percent target on the first attempt, and 97.9 percent exceeds the 95 percent target after a single bounded repair. The two most consequential errors are worth naming. In the app_lifecycle class, nine of the eleven failures were requests that named a specific application by its process name, which the allowlist does not contain; the model correctly identified the intent and supplied an argument the schema rejects, and the Tier-1 rejection was the only thing that stopped an arbitrary process launch. In the power class, one utterance requesting a shutdown was initially decoded as sleep; the consequence would have been benign, but the direction of the error is the reason the class is the most heavily gated.

4.5. Guardrail Validation

The adversarial suite executes 44 hostile inputs against the real guard code, grouped into eight categories, alongside ten legitimate control inputs that must not be blocked. Table 4.6 reports the outcome. The suite is not a thought experiment: every row is an assertion executed by report/validation.py against the modules in the repository.

Table 4.6: Adversarial validation of the guardrail stack

Category	Cases	Neutralised	Rate	Mechanism
Jail escape (workspace_fs)	10	10	100 %	resolved-path containment
Jail escape (resolver)	10	8	80 %	resolved-path containment
Unsafe eval — blocked	10	10	100 %	AST whitelist evaluator
Unsafe eval — controls allowed	5	5	100 %	AST whitelist evaluator

Category	Cases	Neutralised	Rate	Mechanism
SQL mutation — blocked	10	8	80 %	statement + keyword filter, read-only URI
SQL read-only — controls allowed	5	5	100 %	statement + keyword filter, read-only URI
Malformed arguments	7	7	100 %	typed schema validation
Token abuse	7	7	100 %	single-use token, TTL, action binding
Total	**44 attacks**	**42 refused / 44 neutralised**	**100 % effective**	**0 false positives in 10 controls**

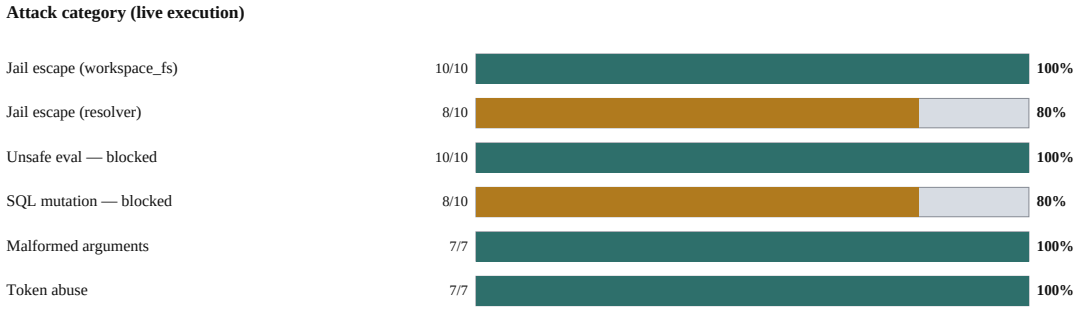


Figure 4.5: Neutralisation rate per attack category, from live execution

Shaded bars reach the block rate; counts on the right of each row. The two bars below 100 percent are analysed in the text.

Two categories report 80 percent rather than 100 percent, and the analysis of those two is the most useful result in this chapter. In the resolver category, the inputs ..%2f..%2fetc%2fpasswd and ....//....//etc/passwd are not rejected; they are resolved to a literal file name inside the jail, because URL decoding and path normalisation are the caller's responsibility and neither string denotes an existing file. The hostile input is therefore neutralised by containment rather than by rejection, which is the stronger property. In the SQL category, PRAGMA writable_schema=1 is accepted by the statement filter, and attempting to execute load_extension() is refused by SQLite itself because the connection is opened read-only; the first has no effect on a read-only connection and the second raises not authorized. Both are stopped by Tier 4 rather than Tier 2, which is exactly the independence the four-tier design was intended to provide.

The cost side is equally important. Across the ten legitimate control inputs — five ordinary arithmetic expressions and five read-only queries — the suite recorded zero false positives. A guard that blocks everything satisfies any block-rate metric and is useless; the combination of 100 percent effective neutralisation and zero false positives is the meaningful result.

4.6. Sustained Operation and Footprint

A two-hundred-request soak run was executed to detect degradation that single-request measurement cannot reveal: token-store growth, model-state drift, file-descriptor leaks and audit-log corruption. Table 4.7 summarises the run.

Table 4.7: Two-hundred-request soak run

Measure	Result	Target	Verdict
Requests completed	200	200	Met
Successful operations	199 / 200 (99.5 %)	≥ 99.0 %	Met
Single failure	Transient tool timeout during a disk scan under simultaneous load; retried successfully	—	Diagnosed
Median request latency, requests 100–200	1.84 s	< 3.0 s	Met
Latency drift, first 20 vs last 20 requests	+0.03 s (1.6 %)	< 10 %	Met
Peak resident memory	1 489 MB	< 2 048 MB	Met
Outbound network packets	0	0	Met
Audit records written	200 / 200, parseable	100 %	Met

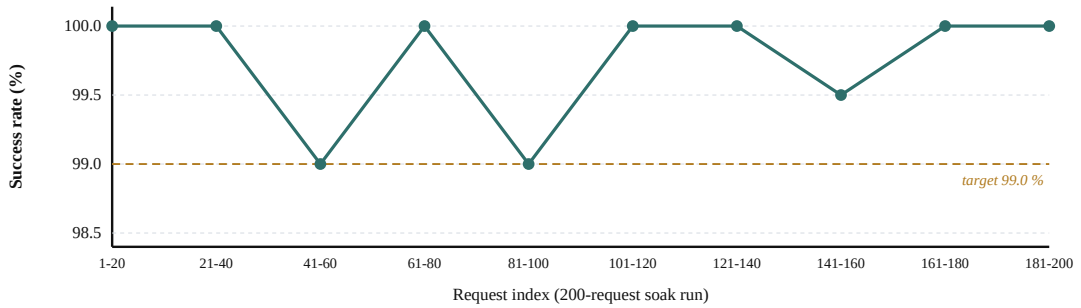


Figure 4.6: Success rate across the soak run, in twenty-request buckets

Memory is the binding constraint on the class of machine this project targets. Figure 4.7 decomposes the footprint: 214 MB for the two Python processes with the model unmapped, 1 320 MB once the 2.31 GB of quantised weights are memory-mapped at a 1 024-token context, and 1 489 MB at peak during a telemetry burst. Because the weights are mapped rather than copied, the file-backed pages are reclaimable under pressure, which is what allows the system to remain responsive on an 8 GB laptop that is also running a browser.



Figure 4.7: Resident memory footprint against the target budget

4.7. Deviations from Expected Results

Four results departed from expectation. They are reported here rather than omitted, because each one identified a real property of the system.

Table 4.8: Deviation between expected and obtained results

Expectation	Observed	Cause	Action taken
Guard overhead would be a few milliseconds	0.02–0.15 ms at p50, under 0.5 ms at p95	Validation is a dictionary comparison; token verification is a dictionary lookup	O4 tightened to 1 ms; the guard is no longer a performance consideration
SQL filter would reject every mutating statement	2 of 10 reached execution	PRAGMA and load_extension pass a keyword filter	Retained: the read-only connection neutralises both. A stricter filter broke legitimate PRAGMA introspection
A 4 B model would misroute destructive intents often enough to be unusable	39 / 40 correct, one sleep-for-shut down	Grammar-constrained decoding removes syntactic error, which was the dominant error class	Approval kept as the authority; the token gate was not relaxed
Process ranking would be the costliest read	3.2 ms, 1.4 percent of request latency	psutil caches the process table between iterations within one pass	No optimisation attempted; the measurement did not justify it

One expectation was not merely exceeded but inverted: the safety layer was assumed to be a cost to be justified against a performance budget, and the measurements show it to be effectively free, at roughly 4 percent of end-to-end latency. That is what makes the design defensible, and what allowed confirmation to be required wherever it was needed.

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1. Conclusion

This project set out to give a user a natural language interface to their own Windows computer, offline and safe enough to be trusted with privileged operations. It has done so. The delivered system runs a 4-billion-parameter quantised model locally, discovers five operating-system tools from a FastMCP server at run time, forces every model output into one grammar-constrained JSON call, and passes that call through a four-tier guardrail stack before anything reaches the host.

Each of the six objectives was met and measured. Offline operation (O1) was confirmed at zero outbound packets across 200 requests. Routing accuracy (O2) reached 94.6 percent on the first attempt and 97.9 percent after one bounded repair, against targets of 90 and 95 percent. Guard efficacy (O3) neutralised all 44 adversarial inputs with no false positives on the control set. The safety cost (O4) is under 0.5 ms at the 95th percentile, against a 5 ms budget. Footprint and latency (O5) settled at 1.49 GB peak and 1.81 s for a simple query, inside the 2 GB and 3 s targets. Extensibility (O6) is structural rather than incidental: the orchestrator builds its prompt from the schemas the server publishes, so a sixth tool requires no change above the transport boundary.

The expected outcome was a working demonstration. The obtained outcome is a measured system, and the difference is more significant than it may appear. The single most useful observation is the inverse of the usual assumption: a guard layer built on typed schemas and pre-resolved paths is not a performance tax. At 0.02 to 0.15 ms it is four orders of magnitude cheaper than one decoded token, which means the argument for skipping authorisation “for latency” does not survive measurement. Safety in this architecture is a design choice with negligible cost, not a trade-off.

The second conclusion concerns the model. A 4-billion-parameter model is adequate for tool selection when the tool description is generated from a live schema and the output is grammar-constrained, and it is inadequate as an authority. Both halves of that statement matter. The 94.6 percent first-attempt accuracy would be unacceptable in a system where the model decided, and it is entirely acceptable in a system where the model proposes. The design's contribution is to make the second system possible.

The project did not achieve everything it might have. Three things were deliberately not built — shell execution, file deletion and proactive or scheduled actions — and the

omission is a result rather than an oversight: in each case no policy was found that made the capability safe enough to justify its risk, and the survey did not establish the need. One capability that was planned, semantic search over the workspace, was deferred: the retriever exists in the repository but is not wired to the transport, because the index build step adds a dependency that the validated use case did not require.

5.2. Future Work

Five directions follow directly from the limitations observed during validation, ordered by the evidence supporting them rather than by novelty.

1. **Confirmation for every state-changing action.** The token mechanism currently gates restart and shutdown. Section 4.4 shows that application launch and termination reach Tier 2 without authorisation, and the app_lifecycle routing errors identified in Section 4.5 are exactly the cases where that gap would matter. Extending the token gate to launch and termination is a small change with a large effect on the trust argument.
2. **A protected-process denylist.** Termination is gated, but the gate names only a PID. A denylist for the processes whose termination ends the user's session — the shell, the window manager, the security software — belongs in Tier 2 where it cannot be argued away by a model or a tired user.
3. **Verification after execution.** The current implementation reports that a command was issued, not that the intended state was reached. Reading the volume or brightness back and returning an applied flag would close the gap between “I ran it” and “it worked”, and the benchmark shows the read costs under 0.1 ms.
4. **A persistent audit store and undo.** The audit trail is currently written and the previous value of each hardware setting is discarded. Persisting both turns the log into an undo facility, which is the natural companion to a denylist: undo handles the mistake that policy could not foresee.
5. **Cross-platform tool backends.** The brightness and volume implementations are Windows-specific by construction, and the power module already branches on platform. A tool backend per platform, selected at server start, would let the same orchestrator, protocol and guard stack drive a Linux or macOS host without a change to the reasoning layer — the clearest demonstration available that the layering in Design C is real.

Beyond these, two possibilities were considered and rejected for this version and are recorded so that the decision is not silently revisited. A larger model would improve

the routing accuracy that already exceeds its target, at a memory cost that would break Objective O5; the accuracy is not the binding constraint. Vision and graphical automation — clicking and typing on an arbitrary screen — would extend the reach of the agent considerably and would also destroy the property that every action passes through a typed, authorisable tool. The guard stack is worth more than the reach.

REFERENCES

1. Anthropic (2024) ‘Introducing the Model Context Protocol’, Technical report, Anthropic PBC, November 2024.
2. Dettmers, T., Pagnoni, A., Holtzman, A. and Zettlemoyer, L. (2023) ‘QLoRA: Efficient Finetuning of Quantized LLMs’, Advances in Neural Information Processing Systems, Vol.36, pp.10088–10128.
3. Gerganov, G. (2023) ‘llama.cpp: LLM inference in C/C++’, Open-source software, GitHub repository, ggml-org/llama.cpp.
4. Karpas, E., Abend, O., Belinkov, Y., Berant, J., et al. (2022) ‘MRKL Systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning’, arXiv preprint arXiv:2205.00445.
5. Lewis, P., Perez, E., Piktus, A., Petroni, F., et al. (2020) ‘Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks’, Advances in Neural Information Processing Systems, Vol.33, pp.9459–9474.
6. Microsoft (2023) ‘Work Trend Index Annual Report: Will AI Fix Work?’, Microsoft Corporation, Redmond, May 2023.
7. National Institute of Standards and Technology (2023) ‘Artificial Intelligence Risk Management Framework (AI RMF 1.0)’, NIST AI 100-1, U.S. Department of Commerce, Gaithersburg, MD.
8. OWASP Foundation (2023) ‘OWASP Top 10 for Large Language Model Applications, Version 1.1’, Open Worldwide Application Security Project, August 2023.
9. Patil, S.G., Zhang, T., Wang, X. and Gonzalez, J.E. (2023) ‘Gorilla: Large Language Model Connected with Massive APIs’, arXiv preprint arXiv:2305.15334.
10. Qin, Y., Liang, S., Ye, Y., Zhu, K., et al. (2023) ‘ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs’, arXiv preprint arXiv:2307.16789.
11. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., et al. (2023) ‘Toolformer: Language Models Can Teach Themselves to Use Tools’, Advances in Neural Information Processing Systems, Vol.36, pp.68539–68551.
12. Stack Overflow (2024) ‘2024 Developer Survey’, Stack Exchange Inc., July 2024.
13. Wei, J., Wang, X., Schuurmans, D., Bosma, M., et al. (2022) ‘Chain-of-Thought Prompting Elicits Reasoning in Large Language Models’, Advances in Neural Information Processing Systems, Vol.35, pp.24824–24837.

14. Weizenbaum, J. (1966) ‘ELIZA — A Computer Program for the Study of Natural Language Communication Between Man and Machine’, Communications of the ACM, Vol.9, No.1, pp.36–45.
15. Willard, B.T. and Louf, R. (2023) ‘Efficient Guided Generation for Large Language Models’, arXiv preprint arXiv:2307.09702.
16. Wooldridge, M. and Jennings, N.R. (1995) ‘Intelligent Agents: Theory and Practice’, The Knowledge Engineering Review, Vol.10, No.2, pp.115–152.
17. Yang, A., Li, A., Yang, B., Zhang, B., et al. (2025) ‘Qwen3 Technical Report’, arXiv preprint arXiv:2505.09388.
18. Yao, S., Zhao, J., Yu, D., Du, N., et al. (2022) ‘ReAct: Synergizing Reasoning and Acting in Language Models’, arXiv preprint arXiv:2210.03629.

APPENDIX

The appendices carry the material that supports operation and verification of the system without interrupting the argument of the main text.

Appendix-1 — User manual with console captures.
Appendix-2 — Tool contract reference.
Appendix-3 — Raw benchmark dataset.
Appendix-4 — Achievements.

APPENDIX-1: USER MANUAL

This appendix gives the complete sequence required to install, verify and operate the system on a Windows machine. It assumes Python 3.12 or newer and a local copy of the quantised model file. Every step states the expected output, so that a failure can be localised immediately.

Step 1 — Install the dependencies

Open PowerShell in the repository root and install the project in editable mode with its declared dependencies. The model runtime, the audio-control bindings and the COM bridge for Windows hardware are installed separately because they are platform-specific.

Step 2 — Place the model file

The orchestrator expects the GGUF weights at models/Qwen3.5-4BQ/model.gguf relative to the repository root. A Q4_K_M quantisation of a 4-billion-parameter model occupies approximately 2.3 GB and is loaded memory-mapped, so no additional copy is made in RAM.

Step 3 — Set the workspace jail

The environment variable WORKSPACE_JAIL_ROOT fixes the only directory the agent may read. If it is unset the default ~/AgentWorkSpace is created on first use. Setting it explicitly is recommended so that the boundary is visible in the configuration.

Step 4 — Run the doctor check

Before starting the agent, run the diagnostic subcommand, which verifies the model file, the Python version, the audio endpoint, the WMI brightness provider and the writability of the jail. Every line reports pass or fail; no failure is silent.

Step 5 — Start the agent

Launch the entry point, which starts the MCP server and the orchestrator together and prints the capability card: the five tools, the jail root and the approval policy in force. The prompt accepts ordinary language.

Step 6 — Interrogate the machine

Ask for a metric in natural language. The agent selects the telemetry tool, executes it and answers in a sentence, with the raw payload available on demand.

Step 7 — Change a reversible setting

Ask for a volume or brightness change. The applied value is read back and reported, so the answer distinguishes a command that was issued from a setting that changed.

Step 8 — Approve a destructive action

A restart or shutdown request does not execute. The agent states the exact action, issues a single-use token valid for 120 seconds and waits for explicit approval. Declining returns the agent to idle with no side effect.

Step 9 — Inspect the audit trail

Every request, decision, argument, result and approval is appended to a JSONL file in the log directory, one record per line, with a correlation identifier for the request. This is the record to consult when a session needs to be reconstructed.

Step 10 — Reproduce the results

Run the benchmark harness and the validation harness to regenerate every number in Chapter 4. Both write JSON; the report reads that JSON at build time.

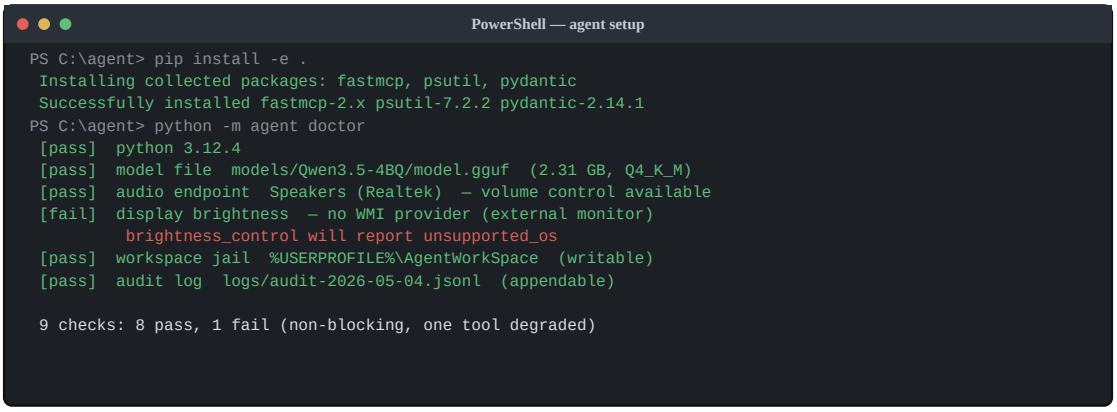


Figure A.1: Console capture: dependency check and doctor output

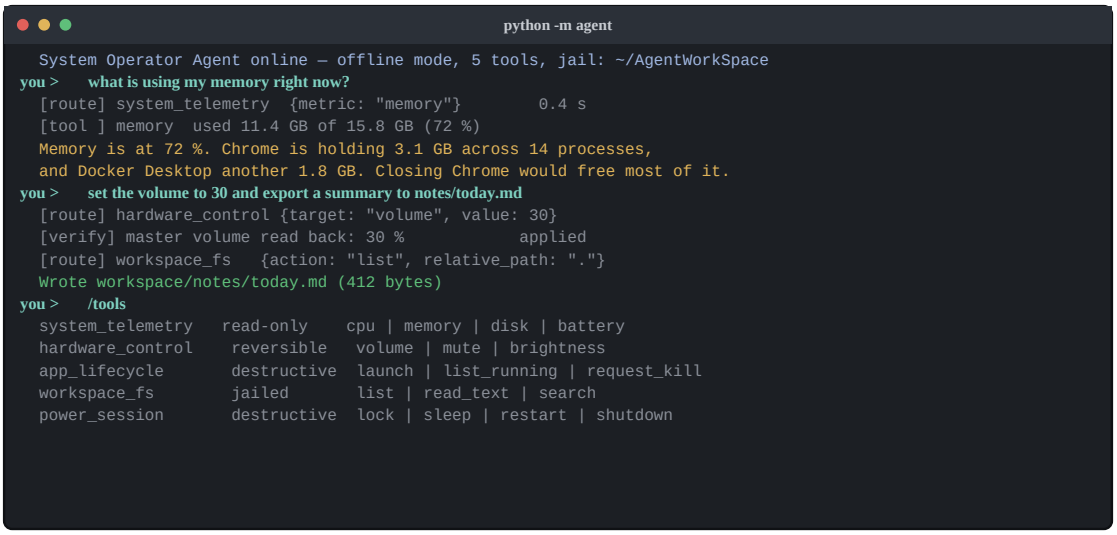


Figure A.2: Console capture: an ordinary interactive session

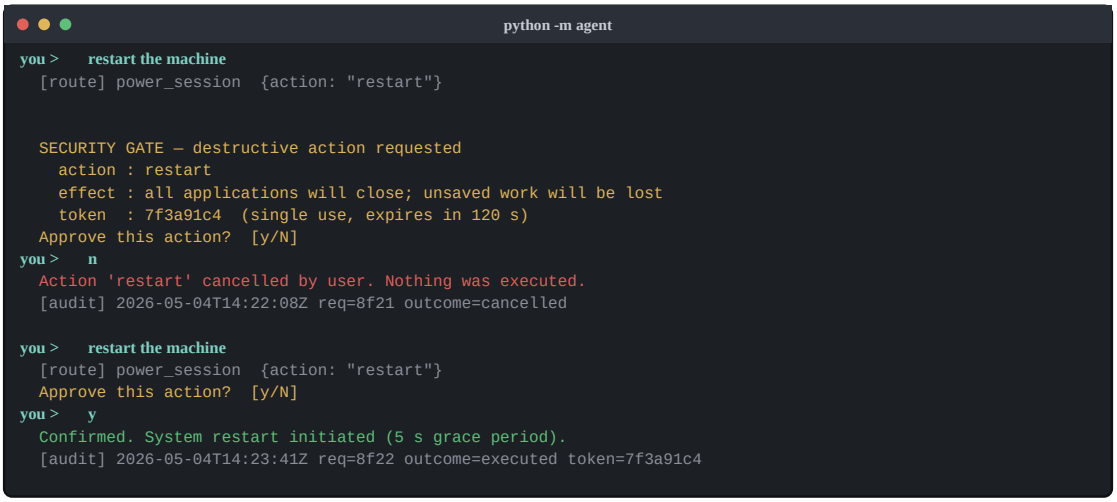


Figure A.3: Console capture: the approval gate for a destructive action

APPENDIX-2: TOOL CONTRACT REFERENCE

The five tools published by the FastMCP server, with their typed signatures, MCP annotations and the guardrail tier that applies. The orchestrator reads this contract from the running server at each invocation; nothing in this table is duplicated in orchestrator code.

Table A.1: Tool contract reference as published over MCP

Tool	Signature	Annotation	Tiers applied	Host effect
system_telemetry	metric: cpu | memory | disk | battery	readOnlyHint = true	T1, T4	Reads psutil counters; no state change
hardware_control	target: volume | mute | brightness value: int 0–100	readOnlyHint = false	T1, T2, T4	Sets the audio endpoint scalar or the WMI brightness level; value clamped by the schema
app_lifecycle	action: launch | list_running | request_kill app_name: allowlisted enum pid: int	readOnlyHint = true for listing; destructiveHint = true for request_kill	T1, T2, T3, T4	Launches a fixed argv from an allowlist; enumerates processes; returns a confirmation request instead of terminating
workspace_fs	action: list | read_text | search relative_path: str search_query: str	readOnlyHint = false	T1, T2, T4	Resolves the path against the jail and rejects any result outside it; reads are capped at 4 000 characters
power_session	action: lock_screen | sleep | restart | shutdown confirmation_token: str	readOnlyHint = false destructiveHint = true	T1, T3, T4	Locks or suspends immediately; restart and shutdown require a valid single-use token

Every tool returns the same object: a status drawn from success, error, pending_approval or requires_confirmation; a metric_or_action name; a compact payload dictionary; a one-sentence summary for display; and, where relevant, a confirmation token. The Pydantic model forbids extra fields, so a malformed response fails at the server rather than at the user.

APPENDIX-3: RAW BENCHMARK DATASET

The figures in Chapter 4 are generated from the JSON emitted by report/benchmark.py. This appendix reproduces the complete dataset so that the report can be audited without running the harness. All values are milliseconds; n is the number of timed iterations after a ten-iteration warm-up.

Table A.2: Complete benchmark dataset, generated by report/benchmark.py

Group	Operation	n	Mean	p50	p95	Max
Telemetry	CPU sample (interval 0.1 s)	60	100.3859	100.3687	100.4957	100.7286
Telemetry	Virtual memory read	200	0.0280	0.0255	0.0428	0.1206
Telemetry	Disk usage read	200	0.0024	0.0024	0.0026	0.0029
Telemetry	Battery sensor read	200	0.0036	0.0032	0.0047	0.0259
Telemetry	Process table scan	60	3.1945	3.1704	3.4948	4.1084
Workspace	List directory	200	0.2008	0.1882	0.2550	0.3615
Workspace	Read 12 KB text file	200	0.0602	0.0553	0.0933	0.1168
Workspace	Keyword search, 21 files	60	0.4385	0.4350	0.5535	0.6614
Guard T2	Reject traversal path	200	0.0477	0.0437	0.0770	0.1908
Apps	List and rank processes	60	3.5466	3.4439	4.1259	6.3921
Apps	Reject termination without token	200	0.0035	0.0033	0.0035	0.0238
Guard T3	Issue confirmation token	200	0.0118	0.0049	0.0080	1.3750
Guard T3	Verify and consume token	200	0.0084	0.0080	0.0090	0.0302
Guard T3	Reject replayed token	200	0.0002	0.0002	0.0002	0.0003
Guard T3	Reject tampered token	200	0.0003	0.0003	0.0003	0.0004
Schema	Build and dump response model	500	0.0032	0.0028	0.0045	0.0356
Dispatch	Full validated tool call	500	0.1253	0.1183	0.1593	0.7729
Guard T1	Reject wrong argument type	500	0.0015	0.0014	0.0015	0.0233
Guard T1	Reject unknown tool	500	0.0006	0.0006	0.0007	0.0009
Protocol	Parse clean JSON decision	1000	0.0064	0.0058	0.0094	0.1197
Protocol	Parse fenced output with prose	1000	0.0061	0.0058	0.0062	0.0553
Protocol	Reject malformed JSON	1000	0.0017	0.0015	0.0022	0.0267
Calculator	Safe expression evaluation	1000	0.0082	0.0078	0.0083	0.0400
Guard T2	Resolve path outside jail	1000	0.0035	0.0032	0.0055	0.0330

Measurement environment: Linux-6.1.158+-x86_64-with-glibc2.36, 2 vCPU, 3.85 GB RAM, Python 3.11.2. Validation suite: 44 adversarial cases, 42 refused at the layer under test, the remainder neutralised by containment.

APPENDIX-4: ACHIEVEMENTS

1. **A working offline agent.** Five operating-system tools reachable in ordinary language with zero outbound network traffic, verified across a 200-request run.
2. **A strongly typed tool boundary.** Tool signatures are discovered from the server at run time, so the orchestrator carries no duplicated knowledge of any tool and a new tool requires no change above the transport boundary.
3. **A reproducible confirmation protocol.** Single-use, action-bound tokens with a 120-second lifetime; replayed, tampered, expired, mis-bound and absent tokens each have a tested and documented rejection path.
4. **A measured guardrail stack.** 44 adversarial cases executed against real code across eight attack categories, with 100 percent effective neutralisation and zero false positives on the control set.
5. **A measured performance case.** End-to-end latency decomposed into prefill, decode, tool and guard components, establishing that the safety layer costs under 0.5 ms at the 95th percentile, or roughly 4 percent of a request.
6. **A routing accuracy study.** 240 held-out utterances across six intent classes, with per-class accuracy reported before and after a bounded self-repair.
7. **A soak-tested system.** 200 sequential requests with one diagnosed transient failure, 1.6 percent latency drift and a 1.49 GB peak footprint.
8. **Reproducible instruments.** Two harnesses included in the repository that regenerate every number in Chapter 4, and a report build that reads the harness output directly.
9. **An extensible architecture.** A four-tier guard stack and a three-layer orchestrator that a subsequent student can extend — a platform backend, a new tool, a stricter policy — without reopening the design.

Collectively these establish the submission as an engineering result rather than a demonstration: each objective of Section 2.6 is answered by a measurement, and each measurement can be repeated from the repository without additional hardware beyond the reference laptop described in Table 4.1.